Question 1

Navigate to the directory that contains password wordlists used for security testing tools.

Approach

I navigated to the directory that stores password wordlists commonly used in penetration testing tools.

Commands

- `cd` → changes directory
- `pwd` → confirms current location
- `ls` → lists files inside directory

Output

STEP 1 : Navigate to wordlists directory

```
┌──(aman㉿kali)-[~]
└─$ cd /usr/share/metasploit-framework/data/wordlists
```

STEP 2 : Verify current directory

```
┌──(aman㉿kali)-[/usr/share/metasploit-framework/data/wordlists]
└─$ pwd
/usr/share/metasploit-framework/data/wordlists
```

STEP 3 : List available wordlists

```
┌──(aman㉿kali)-[/usr/share/metasploit-framework/data/wordlists]
└─$ ls
adobe_top100_pass.txt          db2_default_userpass.txt                  http_default_users.txt  mirai_user_pass.txt             postgres_default_pass.txt      sensitive_files.txt                      unix_passwords.txt
av_hips_executables.txt        db2_default_user.txt                      http_owa_common.txt     mirai_user.txt                  postgres_default_userpass.txt  sensitive_files_win.txt                  unix_users.txt
av-update-urls.txt             default_pass_for_services_unhash.txt      idrac_default_pass.txt  multi_vendor_cctv_dvr_pass.txt  postgres_default_user.txt      sid.txt                                  vnc_passwords.txt
burnett_top_1024.txt           default_userpass_for_services_unhash.txt  idrac_default_user.txt  multi_vendor_cctv_dvr_users.txt root_userpass.txt              snmp_default_pass.txt                    vxworks_collide_20.txt
burnett_top_500.txt            default_users_for_services_unhash.txt     ipmi_passwords.txt      named_pipes.txt                 routers_userpass.txt           superset_secret_keys.txt                 vxworks_common_20.txt
can_flood_frames.txt           dlink_telnet_backdoor_userpass.txt        ipmi_users.txt          namelist.txt                    rpc_names.txt                  telerik_ui_asp_net_ajax_versions.txt     wp-exploitable-plugins.txt
cms400net_default_userpass.txt flask_secret_keys.txt                     joomla.txt              oracle_default_hashes.txt       rservices_from_users.txt       telnet_cdata_ftth_backdoor_userpass.txt  wp-exploitable-themes.txt
common_roots.txt               grafana_plugins.txt                       keyboard-patterns.txt   oracle_default_passwords.csv    sap_common.txt                 tftp.txt                                 wp-plugins.txt
dangerzone_a.txt               hci_oracle_passwords.csv                  lync_subdomains.txt     oracle_default_userpass.txt     sap_default.txt                tomcat_mgr_default_pass.txt              wp-themes.txt
dangerzone_b.txt               http_default_pass.txt                     malicious_urls.txt      password.lst                    sap_icm_paths.txt              tomcat_mgr_default_userpass.txt
db2_default_pass.txt           http_default_userpass.txt                 mirai_pass.txt          piata_ssh_userpass.txt          scada_default_userpass.txt     tomcat_mgr_default_users.txt
```

Key Observations

The directory contain multiple password lists used for brute-force and dictionary attacks.

What I learned

Security tools often store wordlists in predefined directories that can be accessed through the terminal.

Question 2

Display the contents of the password list file using a command-line tool.

Approach

I used a command that prints file content directly in the terminal.

Commands

- `cat` → displays file contents

Output

STEP 1 : View contents of password list

```
┌──(aman㉿kali)-[/usr/share/metasploit-framework/data/wordlists]
└─$ cat password.lst
!@#$%
!@#$%^
!@#$%^&
!@#$%^&*
!boerbul
!boerseun
!gatvol
!hotnot
!kak
!koedoe
!likable
!poes
!pomp
!soutpiel
.net
0
000000
00000000
0007
007
007007
0s
0th
1
10
100
1000
1000s
100s
1022
10s
10sne1
1111
```

Key Observations

The file contains many commonly used passwords.

What I learned

Wordlists contain possible passwords that attackers may test against login systems.

Question 3

View the contains of the password list page by page using a command designed for large files.

Approach

Since the file contains many entries, I used a command that allows scrolling output one screen at a time

Commands

- `more` → displays file content page by page

Output

STEP 1 : View file using more

```
┌──(aman㉿kali)-[/usr/share/metasploit-framework/data/wordlists]
└─$ more password.lst
!@#$%
!@#$%^
!@#$%^&
!@#$%^&*
!boerbul
!boerseun
!gatvol
!hotnot
!kak
!koedoe
!likable
!poes
!pomp
!soutpiel
.net
0
000000
00000000
0007
007
007007
0s
0th
1
10
100
1000
1000s
100s
1022
10s
10sne1
1111
11111
111111
11111111
112233
1212
121212
1213
1214
1225
123
123123
123321
1234
12345
123456
1234567
12345678
```

Key Observation

The output pauses after each screen, allowing controlled reading.

What I learned

Some files are too large to read at once, so paging tools help navigate content efficiently.

Question 4

Use a command that allows flexible navigation through a large file.

Approach

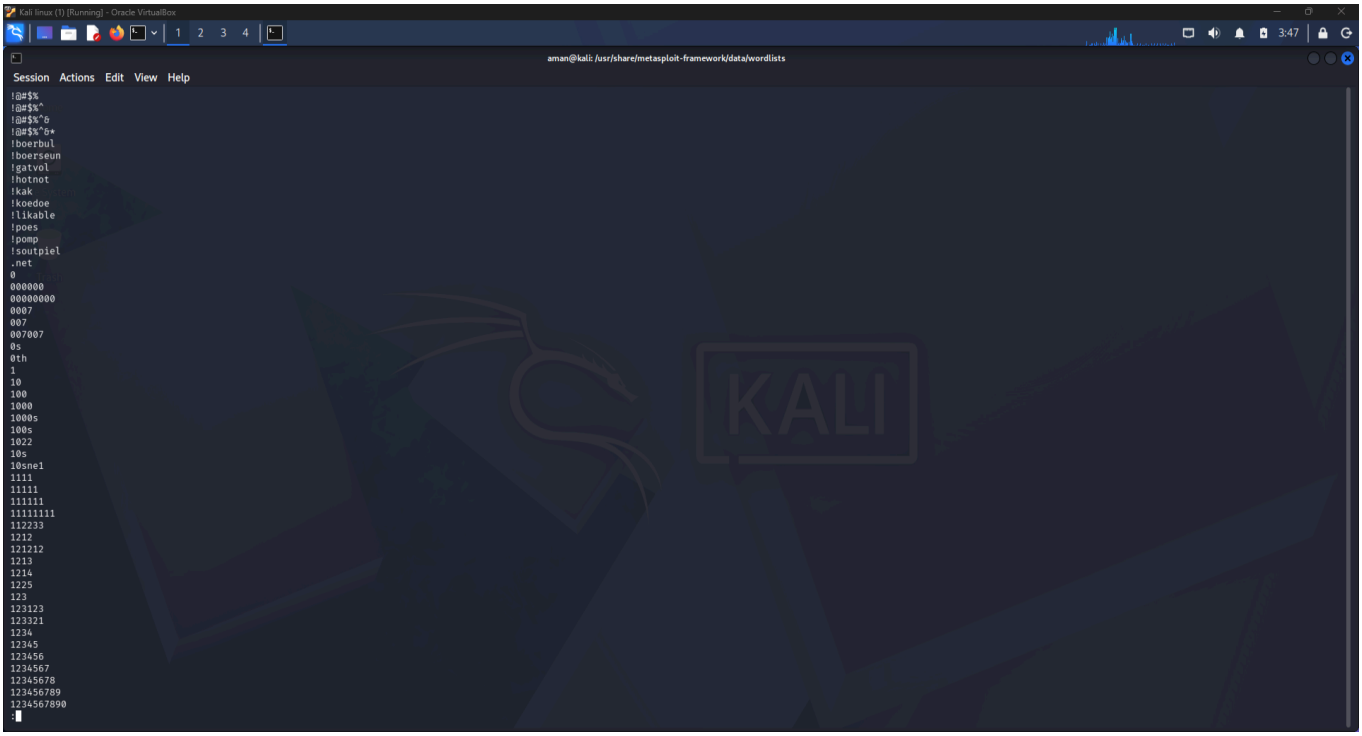
I used a command that allows scrolling up and down within the file.

Commands

- `less` → allows forward and backward navigation

Output

STEP 1 : View file using less



Key Observations

Unlike `more` , this command allows scrolling in both directions.

What I learned

Efficient file viewing is important when analyzing logs or large datasets.

Question 5

Display the password list with line numbers.

Approach

I used a command that adds numbering to each line in the file.

Commands

- `nl` → adds line numbers to file output

Output

STEP 1 : Show numbered lines

```
┌──(aman㉿kali)-[/usr/share/metasploit-framework/data/wordlists]
└─$ nl password.lst
     1  !@#$%
     2  !@#$%^
     3  !@#$%^&
     4  !@#$%^&*
     5  !boerbul
     6  !boerseun
     7  !gatvol
     8  !hotnot
     9  !kak
    10  !koedoe
    11  !likable
    12  !poes
    13  !pomp
    14  !soutpiel
    15  .net
    16  0
    17  000000
    18  00000000
    19  0007
    20  007
    21  007007
    22  0s
    23  0th
    24  1
    25  10
    26  100
    27  1000
    28  1000s
    29  100s
    30  1022
    31  10s
    32  10sne1
    33  1111
    34  11111
    35  111111
    36  11111111
    37  112233
    38  1212
    39  121212
    40  1213
    41  1214
    42  1225
    43  123
    44  123123
    45  123321
    46  1234
    47  12345
    48  123456
    49  1234567
    50  12345678
    51  123456789
```

```
88357  zukor
88358  zulema
88359  zulu
88360  zululand
88361  zunckels
88362  zuney
88363  zuni
88364  zurich
88365  zuurfontein
88366  zuzana
88367  zwaanswyk
88368  zwartkop
88369  zwartkrans
88370  zwaveldspoort
88371  zwavelpoort
88372  zweletemba
88373  zwelethu
88374  zwelibomvu
88375  zwelihle
88376  zwelisha
88377  zwelithini
88378  zwelitsha
88379  zwide
88380  zwieback
88381  zwingli
88382  zworykin
88383  zxc
88384  zxc123
88385  zxcvb
88386  zxcvbn
88387  zxcvbnm
88388  zydeco
88389  zygote
88390  zygotic
88391  zymurgy
88392  zyrtec
88393  zyuganov
88394  zzz
88395  zürich
88396  Ågar
88397  Ångström
88398  éclair
88399  éclairs
88400  éclat
88401  élan
88402  émigré
88403  émigrés
88404  épée
88405  étude
88406  vagrant

┌──(aman㉿kali)-[/usr/share/metasploit-framework/data/wordlists]
└─$
```

Key Observations

Each password entry is assigned a line number.

What I learned

Line numbering helps reference specific entries when analyzing large files.

Question 6

Show only the last 20 entries from the password list.

Approach

I used a command that displays the ending porting of a file.

Commands

- `tail` → displays last lines of file
- `-20` → specifies number of lines

Output

STEP 1 : Display last 20 passwords

```
┌──(aman㉿kali)-[/usr/share/metasploit-framework/data/wordlists]
└─$ tail -20 password.lst
zxcvbnm
zydeco
zygote
zygotic
zymurgy
zyrtec
zyuganov
zzz
zürich
Ågar
Ångström
éclair
éclairs
éclat
élan
émigré
émigrés
épée
étude
vagrant
```

Key Observations

The command quickly shows the final entries without printing the whole file.

What I learned

Viewing specific parts of files saves time when working with large datasets.

Question 7

Filter the password lists to show only entries containing specific numbers.

Approach

I combined two commands using a pipe to filter results containing a pattern.

Commands

- `cat` → displays file content
- `|` → passes output to another command
- `grep` → searches for pattern
- `123` → search keyword

Output

STEP 1 : Search passwords containing 123

```
┌──(aman㉿kali)-[/usr/share/metasploit-framework/data/wordlists]
└─$ cat password.lst | grep 123
123
123123
123321
1234
12345
123456
1234567
12345678
123456789
1234567890
1234qwer
123abc
123go
123qwe
a12345
abc123
abcd123
abcd1234
aki123
asdf1234
chris123
happy123
hello123
help123
jkl123
red123
test123
xxx123
xyz123
zxc123
```

Key Observation

Only passwords containing "123" are displayed.

What I learned

Piping allows combining commands to filter and analyze data efficiently.

Security Insights

This chapter introduced techniques useful in cybersecurity workflows:

- Wordlists are commonly used in password attacks such as brute force and dictionary attacks
- Tools like cat, less, and more help analysts inspect log files and datasets
- Commands like grep help filter important information from large files

- Understanding file content quickly is essential during penetration testing and forensic investigation

---